



Sprint 8 :

Drupal Advanced Development Guide

Réalisé par :

Sara Akharraz

Encadrants :

M.Abdelmajid Bendrif

M.Hamza Bahlaouane

Entreprise :

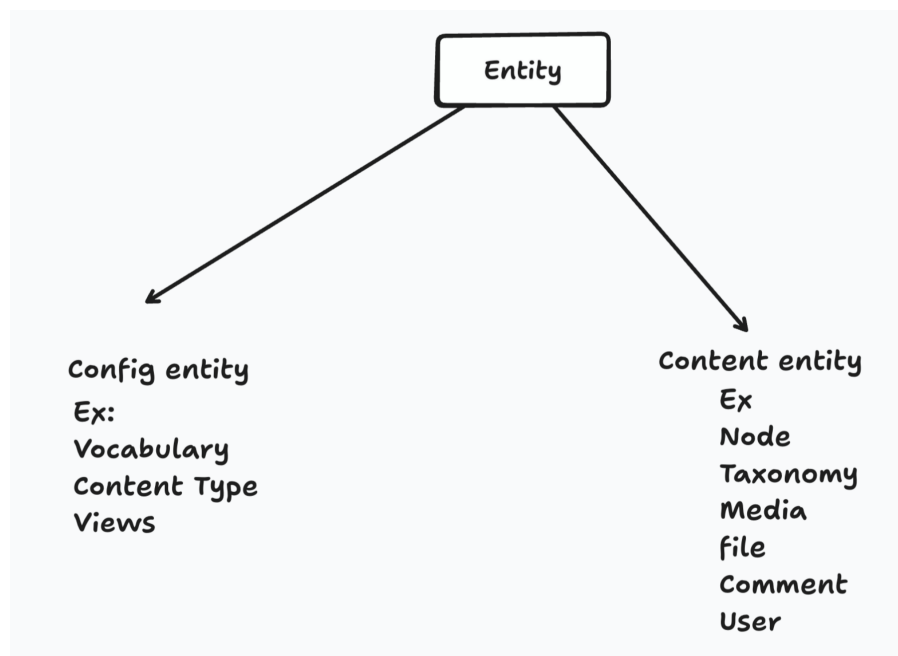
Void

Objectifs :

- Se familiariser avec Entity, Field API Configuration Entities
- Apprendre à travailler avec les Hooks
- Maîtriser la configuration et les fonctionnalités
- Comprendre et utiliser le cache
- Travailler avec la migration de données

Chapitre 1 : Entity, Field API & Configuration Entities

- **Entity API** : Drupal permet de créer, lire, modifier et supprimer des entités. Une *entité* est un objet structuré qui stocke des données. Deux types d'entités :
 - **Content types** : le vrai contenu (articles, pages, etc.)
 - **Config types** : la structure des content types



- **Gestion des entités** :
 - `entityTypeManager()` : manager central pour gérer toutes les entités
 - `getStorage()` : gestion du stockage (charger, sauvegarder, créer, supprimer, interroger)
 - `save()` : enregistre l'entité dans la base de données
- **Problème classique** : Logique conditionnelle basée sur les bundles .

La logique est dispersée : un même champ peut être perçu différemment selon le bundle.

 - Cas 1 : conditions ou switch → dupliquent la logique
 - Cas 2 : `THEME_preprocess_node()` → le thème doit gérer l'affichage, pas la logique métier
 - Cas 3 : hypothèses globales sur la classe Node → pas flexible
- **Solution** : classes de bundle Chaque bundle peut avoir sa propre classe.

Exemple : `ArticleNode` hérite d'une classe de base et définit un comportement spécifique pour les articles.

 - `getShareTitle()` : retourne le titre à partager (ne modifie pas le titre réel)

- `getShareUrl()` : retourne l'URL avec paramètres UTM pour analytics
- Les classes de bundle :
 - Définissent le comportement pour tous les nœuds d'un type (article, page, blog)
 - Héritent de la classe de base de l'entité (**Node**)
 - Chaque nœud conserve ses champs uniques (titre, image, contenu)
- **JSON-LD et schema.org** :
 - JSON-LD : format pour décrire des métadonnées structurées (titre, URL, image)
 - schema.org : vocabulaire standard pour décrire des types de contenu (WebPage, Article, Event, Product...)
 - Exemple de tableau PHP pour partager un article :

```
$data = [
    '@context' => 'https://schema.org',
    '@type' => 'WebPage',
    'headline' => $this->getShareTitle(),
    'url' => $this->getShareUrl()->toString(),
];
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<script type="application/ld+json">{@context:"https://schema.org","@type":"WebPage","headline":"testing new bundle logic | Event bundle","url":"http://\drupal-entity-project.ddev.site/node/53"/>
<link rel="icon" href="/core/themes/olivero/favicon.ico" type="image/vnd.microsoft.icon" />
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<script type="application/ld+json">{@context:"https://schema.org",@type:"WebPage",headline:"bundle-class | Last Update: 2026-03-11","url":"http://drupal-entity-project.ddev.site/vnode/4?utm_source=share\u0026utm_content=
<link rel="icon" href="/core/themes/olivero/favicon.ico" type="image/vnd.microsoft.icon" />
```

Validation et Contrôle d'accès dans Drupal

Validation avec Symfony Validator

- Drupal utilise Symfony Validator : combinaison de **Constraint** + **ConstraintValidator**
- **Constraint** : règle de validation et message d'erreur
- **ConstraintValidator** : code qui vérifie la règle sur les données réelles
- Types de contraintes :
 - **Core** : déjà fournies par Drupal
 - **Custom** : créées pour les entités spécifiques
- Application des contraintes :
 - Lors de la définition des champs (**BaseFieldDefinition**)
 - Sur des entités existantes via **hook_entity_bundle_field_info_alter**
- Exemple : vérifier l'unicité d'un champ → **UniqueFieldConstraint** + **UniqueFieldValueValidator**

Contrôle d'accès

- **Access Control Handler** : classe qui gère toutes les vérifications d'accès
- `checkAccess()` : décide si un utilisateur peut *view*, *update*, *delete*
- **AccessResult** : trois états possibles
 - **allowed** : autorisé explicitement → accès garanti
 - **forbidden** : refusé explicitement → accès bloqué
 - **neutral** : ni autorisé ni refusé → Drupal laisse d'autres modules décider
- Hooks d'accès : `hook_entity_access()` et `hook_ENTITY_TYPE_access()`

Typed Data et Types de Champs dans Drupal

Typed Data

- Drupal utilise **Typed Data** pour gérer les données de manière uniforme.
- Chaque type de donnée est représenté par un plugin géré par le `typed_data_manager`.
- Chaque objet Typed Data contient :
 - La valeur réelle (ex : chaîne, entier, image)
 - Des métadonnées (type, contraintes, longueur, format)
 - Une interface uniforme : `getValue()`, `setValue()`, `getDefinition()`

Types principaux de données

- **Données primitives (Primitive)** : string, integer, email... Méthodes : `getValue()`, `setValue()`
- **Données complexes (Complex)** : contiennent des propriétés nommées (Typed Data). Méthodes : `get()`, `set()`
- **Données en liste (List)** : liste ordonnée d'éléments du même type. Méthodes : `get()`, `set()`, `offsetGet()`

Définition et validation

- Chaque Typed Data a une **data definition** avec métadonnées et contraintes.
- Drupal peut :
 - Valider automatiquement les données (ex : email valide)
 - Sérialiser ou transformer les données (ex : JSON)
 - Fournir une interface uniforme pour manipuler toutes les valeurs
- Avantage : pas besoin de coder la validation et la manipulation pour chaque type.

Widgets de champ (Field Widgets)

- Déterminent comment le champ apparaît dans le formulaire d'édition
- Définition via `#[FieldWidget]` : id, label, types de champs supportés, multi-valeurs possibles

Formatters de champ (Field Formatters)

- Déterminent comment le champ est affiché sur le site (front-end)
- Définition via `#[FieldFormatter]` : id, label, types de champs supportés

Questions :

- Comment utiliser `hook_entity_base_field_info_alter` pour modifier le type d'entité utilisateur et ajouter une contrainte au champ mot de passe ? Votre nouvelle contrainte `PasswordPolicyConstraint` doit utiliser le service `password_policy.validator` du module Password Policy.
Installer et configurer le module Password Policy.

```
composer require drupal/password_policy
```

```

/**
 * Implements hook_entity_base_field_info_alter().
 */
no usages
function content_entity_example_entity_base_field_info_alter(&$fields, EntityTypeInterface $entity_type) {
    if ($entity_type->id() == 'user' && isset($fields['pass'])) {
        $fields['pass']->addConstraint('PasswordPolicyConstraint');
    }
}

```

FIGURE 1 – Enter Caption

Le fichier PasswordPolicyConstraint.php :

```

<?php

namespace Drupal\content_entity_example\Plugin\Validation\Constraint;

use Symfony\Component\Validator\Constraint;

/**
 * Checks password policy.
 *
 * @Constraint(
 *   id = "PasswordPolicyConstraint",
 *   label = @Translation("Password Policy", context = "Validation")
 * )
 */
no usages
class PasswordPolicyConstraint extends Constraint {

    public $message = 'The password does not satisfy the password policy.';

    /**
     * Links this constraint to its validator.
     */
    public function validatedBy() :string {
        return \Drupal\content_entity_example\Plugin\Validation\Constraint\PasswordPolicyConstraintValidator::class;
    }
}

```

Le fichier PasswordPolicyConstraintValidator.php :

```

<?php

namespace Drupal\content_entity_example\Plugin\Validation\Constraint;

use ...

1 usage
class PasswordPolicyConstraintValidator extends ConstraintValidator implements ContainerInjectionInterface {

    protected PasswordPolicyValidatorInterface $validator;

    no usages
    public function __construct(PasswordPolicyValidatorInterface $validator) {
        $this->validator = $validator;
    }

    /**
     * Create from container (inject service).
     */
    public static function create(ContainerInterface $container) {
        return new static(
            $container->get(id: 'password_policy.validator')
        );
    }
}

```

```

/**
 * {@inheritdoc}
 */
public function validate($value, Constraint $constraint) {
    if (empty($value)) {
        return;
    }

    if (!$this->validator->validate($value)) {
        $this->context->addViolation($constraint->message);
    }
}
}

```

- Qu'est-ce que `AccessResult` et comment fonctionne-t-il ?

`AccessResult` indique si un utilisateur peut accéder à une entité :

allowed (autorisé),

forbidden (interdit),

neutral (laisse d'autres modules décider).

- Créer une entité personnalisée implique beaucoup de code répétitif, comment générer rapidement le squelette d'une nouvelle entité ? Utiliser la commande `drupal generate entity` via `Drupal Console` pour créer rapidement le squelette de l'entité.

```

entity:
  entity:bundle-class (bundle-class)      Generate a bundle class for a
                                           content entity.
  entity:configuration (config-entity)    Generates configuration entity
  entity:content (content-entity)         Generates content entity

```

- Comment récupérer la définition d'un champ via le code ?

Utiliser `$entity_type->getFieldDefinitions('bundle')` pour obtenir les définitions de champs d'un bundle.

- Est-il possible de créer plusieurs formatters pour un même type de champ ? Si oui, comment ?

Oui, chaque formatter est une classe différente annotée avec `#[FieldFormatter]`, toutes appliquées au même type de champ.

- Avec `drush`, comment récupérer la configuration (settings) d'un module ?

Utiliser la commande `drush config:get <module.settings>` pour obtenir la configuration d'un module.

Chapitre 2 : Hooks

Naming Hooks

```
hook_module_name_descriptive_name()
```

- Always start with the prefix **"hook_"**
- Include the **machine name of your module**
- Finally, a few words that **describe** the hook

Questions :

- Comment ajouter un nouveau champ de base à un type d'entité existant en utilisant `hook_entity_base_field_info` ?

```
function content_entity_example_entity_base_field_info_alter(array &$fields, EntityTypeInterface $entity_type) {  
  if ($entity_type->id() === 'node') {  
    $fields['subtitle'] = BaseFieldDefinition::create([type: 'string'])  
      ->setLabel(t(string: 'Subtitle'))  
      ->setDescription(t(string: 'A subtitle for the node.'))  
      ->setSettings([  
        'max_length' => 255,  
      ])  
      ->setDisplayOptions([display_context: 'form', [  
        'type' => 'string_textfield',  
        'weight' => 5,  
      ]])  
      ->setDisplayOptions([display_context: 'view', [  
        'label' => 'above',  
        'type' => 'string',  
        'weight' => 5,  
      ]])  
      ->setDisplayConfigurable([display_context: 'form', configurable: TRUE])  
      ->setDisplayConfigurable([display_context: 'view', configurable: TRUE];  
  }  
}
```

Not Secure drupal-entity-project.ddev.site/node/add/article

Manage Shortcuts admin Devel Announcements Edit

Content Structure Appearance Extend Configuration People Reports Help

Authoring information
By admin (1) on 2026-03-12

Promotion options
Promoted to front page

Text format Basic HTML [About text formats](#)

Tags

Enter a comma-separated list. For example: Amsterdam, Mexico City, "Cleveland, Ohio"

Subtitle

A subtitle for the node.

field_share_summary

field_share_image

Create new media

- Quel est le rôle de `hook_update_n` ?

```
function content_entity_example_update_11001() {
  // Ajouter un champ ou mettre à jour une table.
  \Drupal::entityTypeManager()->getStorage('node_type')->create([
    'type' => 'news',
    'name' => 'News',
  ])->save();
}

no usages
function content_entity_example_update_11002() {

  $entity_type_manager = \Drupal::entityTypeManager();
  $entity_definition_update_manager = \Drupal::entityDefinitionUpdateManager();

  $entity_type = $entity_type_manager->getDefinition('event');

  $entity_definition_update_manager->installEntityType($entity_type);
}
```

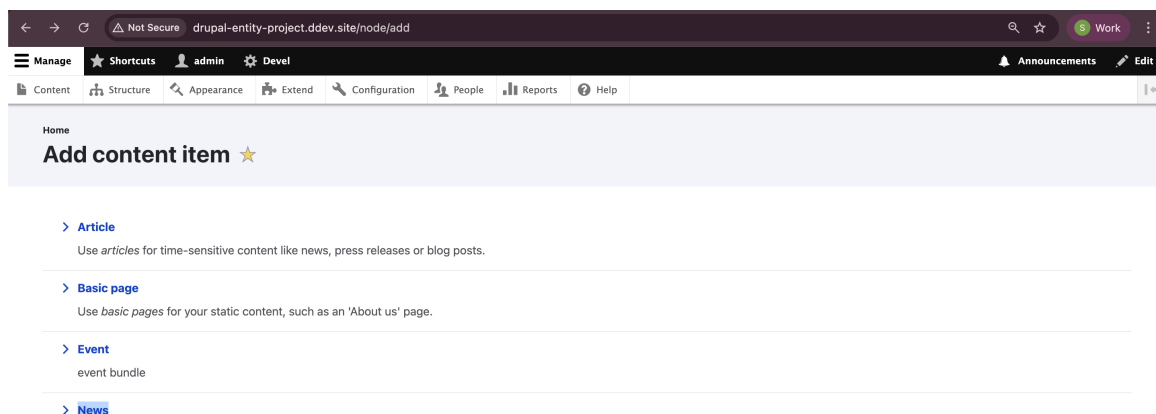
```
→ drupal_entity_project git:(main) × ddev drush updatedb

-----
Module          Update ID   Type          Description
-----
content_entity_example  11001      hook_update_n  11001 -
-----

Do you wish to run the specified pending updates? 
Yes

> [notice] Update started: content_entity_example_update_11001
> [notice] Update completed: content_entity_example_update_11001
[success] Finished performing updates.

→ drupal_entity_project git:(main) ×
```

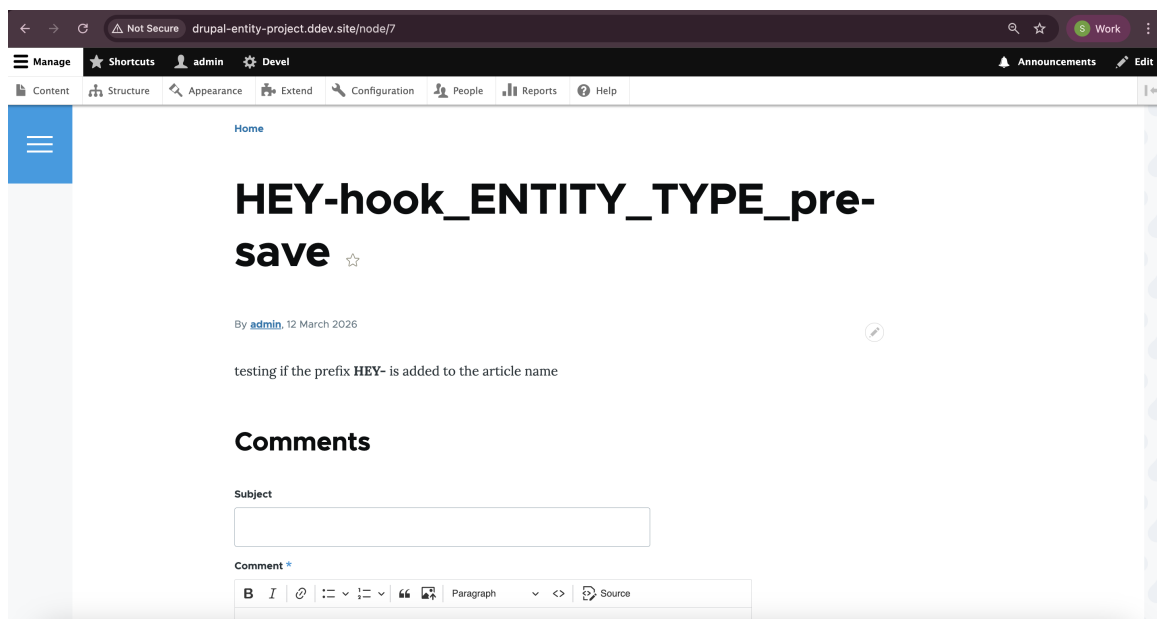


— Quel est le rôle de hook_install?

```
function content_entity_example_install() {
  // Créer des noeuds par défaut :
  \Drupal\node\Entity\NodeType::create([
    'type' => 'article',
    'name' => 'Article',
  ])->save();
}
```

- Comment préfixer tous vos nouveaux nœuds (type : article) avec “HEY-” en utilisant hook_ENTITY_TYPE_presave?

```
function content_entity_example_node_presave(NodeInterface $node) {
  if ($node->bundle() === 'article' && $node->isNew()) {
    $node->setTitle(title: 'HEY-' . $node->getTitle());
  }
}
```



- Quel est le rôle de \$entity->original?
Contient l'état original de l'entité avant la modification. Utile pour comparer les valeurs et détecter les changements.
- Comment peut-on surcharger un Theme Hook fourni par un autre module? **Réponse :** Définir le même hook dans hook_theme() de votre module ou thème et fournir votre propre template ou fonction de rendu.
- Avec hook_theme_suggestions_alter, comment ajouter une nouvelle suggestion de thème pour \$hook === 'user' en fonction du mode d'affichage (view mode)?

```
function content_entity_example_theme_suggestions_alter(array &$suggestions, array $variables, $hook) {
  if ($hook === 'user') {
    $suggestions[] = 'user__custom';
  }
}
}
no usages
function content_entity_example_theme_suggestions_user_alter(array &$suggestions, array $variables) {
  $user = $variables['elements']['#user'];
  if ($variables['elements']['#view_mode'] === 'compact') {
    $suggestions[] = 'user__compact';
  }
}
}
```

Chapitre 3 : Configuration Features

- Deux instances Drupal ont été créées : dev pour le développement et prod pour la production.
- Dans dev, on a installé Devel, créé une taxonomie, un type de contenu avec champs, et défini un rôle avec permissions.
- La configuration a été exportée de dev et importée en prod, en s'assurant que Devel ne soit pas présent et que tout fonctionne.(config split utilisé)
- Avec Features, on a exporté uniquement les parties nécessaires (type de contenu, taxonomie, champs) comme module, puis importé et synchronisé les modifications successives en production.

Différence entre les deux approches

- **Configuration Sync classique** : exporte toute la configuration du site dev et l'importe en production. Risque d'inclure des modules de développement (ex : Devel).

```
→ drupal-dev git:(main) x ddev drush cex
[notice] Differences of the active config to the export directory:
+-----+-----+-----+
| Collection | Config | Operation |
+-----+-----+-----+
| | core.extension | Create |
| | config_split.config_split.dev | Create |
| | config_split.config_split.dev | Delete |
+-----+-----+-----+

The .yaml files in your export directory will be deleted and replaced with...
Yes

[success] Configuration successfully exported to ../config/sync.
../config/sync
→ drupal-dev git:(main) x
```

```
[notice] Synchronized configuration: update system.site.
[notice] Finalizing configuration synchronization.
[success] The configuration was imported successfully.
→ drupal-prod git:(main) x
```

- **Features** : exporte uniquement les éléments nécessaires sous forme de module. Le site production reste propre et les modifications sont mieux contrôlées, ce qui réduit les conflits.

Back to site
Manage
Shortcuts
admin
Devel
Announcements
Edit

Content
Structure
Appearance
Extend
Configuration
People
Reports
Help

General Information

Name

Machine name: my_custom_feature [\[Edit\]](#)
Example: Image gallery (Do not begin name with numbers.)

Description

Provide a short description of what users should expect when they install your feature.

Bundle

Default

Version

Examples: 8.x-1.0, 3.1.4

Components

Search: Clear ☐ Select all: [Expand all](#)

Hide:

- None -

Normal
Added
Auto detected
Conflict

Simple configuration (26)

☒
dblog.settings

Action (17)

Base field override (1)

Block (23)

Home > Administration > Configuration > Development > Features > Edit

Edit ☆

✓
Status message

Package my custom feature written to modules/custom/my_custom_feature.

```

➤ drupal-prod git:(main) x ddev drush pm:install my_custom_feature -y
[success] Module my_custom_feature has been installed.
➤ drupal-prod git:(main) x ddev drush fim my_custom_feature -y

// Do you really want to import my_custom_feature : user.role.blogger?: yes.
➤ drupal-prod git:(main) x

```

Filter

^ Other

☒ my custom feature

Chapitre 4 : Cache API

Principes de base

- **Cacheability** : Décrit dans quelles conditions une donnée peut être mise en cache.
- **Response caching** : Stocke la réponse HTML complète d'une page (ex : Internal Page Cache ou Varnish).
- **Fragment caching** : Met en cache des parties d'une page, comme les render arrays ou les résultats d'API.
- **Cache tags** : Permettent d'invalider le cache lorsqu'un événement spécifique se produit (ex : ajout d'un article).
- **Cache contexts** : Définissent les variations de cache selon le contexte, pour afficher un contenu adapté à l'utilisateur.

- **Cache max-age** : Indique la durée pendant laquelle un élément peut rester en cache.
- **Cache bins** : Séparation du stockage du cache en différents « bins » pour un accès plus efficace.

Cache max-age

- **max-age = 0** : pas de cache, contenu toujours frais.
- **max-age = 60** : contenu mis en cache pendant 60 secondes avant d'être rafraîchi.
- **max-age = CacheableMetadata : :CACHE_PERMANENT** : contenu mis en cache indéfiniment jusqu'à invalidation via les cache-tags.

```
> drupal-dev git:(main) * curl -i http://drupal-dev.ddev.site/api/articles
HTTP/1.1 200 OK
Cache-Control: must-revalidate, no-cache, private
Content-Language: en
Content-Length: 98
Content-Type: application/json
Date: Sun, 15 Mar 2026 17:14:32 GMT
Expires: Sun, 19 Nov 1978 05:00:00 GMT
Server: nginx
X-Content-Type-Options: nosniff
X-Drupal-Cache: UNCACHEABLE (no cacheability)
X-Drupal-Cache-Max-Age: 60
X-Drupal-Dynamic-Cache: UNCACHEABLE (no cacheability)
X-Frame-Options: SAMEORIGIN
X-Generator: Drupal 11 (https://www.drupal.org)

[{"nid":"6","title":"Quia Ullamcorper"},{"nid":"9","title":"Bene Ex"},{"nid":"10","title":"Quia"}]]%
> drupal-dev git:(main) *
```

Différence entre max-age et cache-tags

max-age	cache-tags
Basé sur le temps	Basé sur les événements
Expire après X secondes	Invalidation immédiate lors d'un changement de contenu
Ex : modification d'un nœud n'est pas visible avant expiration	Ex : modification d'un nœud invalide le cache instantanément

```
> drupal-dev git:(main) * curl -i http://drupal-dev.ddev.site/api/articles
HTTP/1.1 200 OK
Cache-Control: must-revalidate, no-cache, private
Content-Language: en
Content-Length: 127
Content-Type: application/json
Date: Sun, 15 Mar 2026 17:27:39 GMT
Expires: Sun, 19 Nov 1978 05:00:00 GMT
Server: nginx
X-Content-Type-Options: nosniff
X-Drupal-Cache: UNCACHEABLE (no cacheability)
X-Drupal-Cache-Tags: node:6,node:9,node:10
X-Drupal-Dynamic-Cache: UNCACHEABLE (no cacheability)
X-Frame-Options: SAMEORIGIN
X-Generator: Drupal 11 (https://www.drupal.org)

[{"nid":"6","title":"Quia Ullamcorper"},{"nid":"9","title":"Bene Ex"},{"nid":"10","title":"Quia_edit_for_cache_max_age_again"}]]%
> drupal-dev git:(main) *
```

- Une nouvelle route `/api/articles` a été créée pour retourner en JSON trois nœuds spécifiques avec leurs `nid` et `title` en utilisant `entityQuery`.
- Le cache a été testé avec **max-age** puis avec **cache-tags** pour observer le rafraîchissement des données lors de la modification du titre d'un nœud. Les **cache-tags** permettent une actualisation immédiate.

Chapitre 5 : Data Migration

Les modules de la migration :

```
composer require drupal/migrate_plus drupal/migrate_tools drupal/migrate_source_csv
```

```
drush en migrate migrate_plus migrate_tools migrate_source_csv -y
```

- **migrate** → Framework de migration fourni par Drupal Core.
- **migrate_plus** → Permet de définir les migrations avec des fichiers YAML.
- **migrate_tools** → Fournit des commandes Drush comme **migrate-import**.
- **migrate_source_csv** → Permet d'utiliser un fichier CSV comme source de données.

[Quel est le rôle de `migration\_lookup` ?](#)

`migration_lookup` permet de récupérer l'identifiant d'une entité déjà importée dans une autre migration. Il est souvent utilisé pour créer des relations entre entités (par exemple lier un nœud à un utilisateur ou une taxonomie déjà migrée).

[Que faire si la source de données n'est pas un fichier CSV mais une base de données MySQL ?](#)

Utiliser un plugin de source adapté, comme **SqlBase** ou les plugins fournis par le module **Migrate Plus**, pour lire directement les données depuis une base de données.

[Comment annuler \(rollback\) une migration ?](#)

Utiliser la commande Drush : `drush migrate:rollback <migration_id>` pour supprimer les données importées par la migration.

[Comment traiter une donnée avant l'import \(ex : limiter une chaîne à 10 caractères\) ?](#)

Utiliser un plugin de processus dans la section **process** de la migration, par exemple **substr** pour limiter la longueur de la chaîne à 10 caractères.